Aero-Grid AI Project Report

Track 5: Urban Air Quality Intelligence & Smart City Intervention
Case Study: Delhi NCR Atmospheric Dispersion & Prioritized Enforcement Dashboard

---

1. Executive Summary

Aero-Grid AI is a full-stack smart city platform designed to move Delhi NCR's environmental response from reactive monitoring to proactive source-level intervention. By fusing simulated CAAQMS ground sensors, MODIS crop fire counts, meteorological wind vector analyses, and diurnal traffic congestion, the system computes geospatial source attribution, generates hyperlocal AQI predictions 72 hours in advance, simulates policy regulations inside a 'What-If' sandbox, and dispatches prioritized enforcement actions directly to inspectors.

2. Case Study: The Delhi NCR Air Quality Crisis

Delhi NCR faces a compound air quality crisis, particularly in winter, where four major factors interact:

- **Vehicular Congestion:** High traffic densities at nodes like ITO and Dhaula Kuan cause localized fine particulate ($PM_{2.5}$) and NO_x accumulation.
- **Industrial Emissions:** Smog stacks in manufacturing hubs (e.g., Bawana, Shadipur) emit high sulfur and volatile particles.
- **Agricultural Smoke:** Regional crop stubble burning in Punjab and Haryana is carried into the Delhi basin by prevailing North-West winds.
- **Thermal Inversion:** Cold boundary layers trap particulates close to the ground, preventing vertical dispersion.

Aero-Grid AI fills the gap by providing the intelligence layer necessary to identify exactly which source is causing a spike at a specific location, allowing surgical local interventions instead of blind economic shutdowns.

3. Product Features & Problem-Solving Matrix

Feature	What it Does	Problem Solved	Smart City Impact
GIS Vector Map	Visualizes sensor circles, wind vector lines, and crop smoke overlays.	Dashboards are typically numbers, lacking spatial context.	Provides a live visual radar of the path of dispersion.
Source Attribution	Calculates % contributions of traffic, factories, biomass, and dust.	Blanket shutdowns hurt the economy because the local root cause is unknown.	Allows target-specific control (e.g. water sprays vs factory caps).
72h Forecasting	Forecasts AQI curves based on meteorological forecasts.	Municipal responses are reactive (acting after air goes toxic).	Provides a 3-day head start to schedule diversions or construction stops.
What-If Sandbox	Simulates policy impacts (e.g., Odd-Even, construction bans) on future AQI.	Environmental policies are made on guesswork with no pre-evaluated metrics.	Enables testing of controls in a simulator to optimize trade-offs.
Enforcement Dispatch	Flags hotspot anomalies and generates prioritized inspector tasks.	Significant delay between alert triggers and inspector dispatch.	Speeds up reaction times to localized pollution violations.
Health Advisory	Translates safety directives into English, Hindi, and Punjabi.	Health alerts do not reach all demographics or offer regional guidelines.	Directs schools and pediatric wards to safety protocols.

4. Technological Architecture & Workflow

Aero-Grid AI is architected as a modular full-stack application. In production, the React client and FastAPI server run in a single container. CORS configurations are bypassed as the FastAPI router mounts the static frontend output at root, presenting a zero-configuration deployment.

Core Architecture Path:

1. **React + TypeScript Frontend:** Runs on the client's browser, managing state adjustments (map hover selections, policy sandbox toggles). Requests are pushed to the Python API.
2. **FastAPI Router:** Serves as the asynchronous REST gateway. Receives telemetry queries and policy simulation payloads, validating inputs via Pydantic schemas.
3. **Simulation Core (NumPy):** Runs physics-based calculations in Python. It executes vector mappings and chemical fingerprinting ratios, returning the baseline vs mitigated forecasts.
4. **Docker Container:** Packs the compiled React `dist` bundle inside a slim Python container, running on port `7860` for Hugging Face Spaces compatibility.

5. API Endpoints Contract

GET /api/stations: Retrieves monitoring stations list with base telemetry.

GET /api/attribution: Fuses weather variables to calculate source contribution percentages.

POST /api/simulate: Simulates the 72-hour forecast timelines. Example Payload:

```
{
  "station_id": "anand_vihar",
  "forecast_hours": 72,
  "policies": {
    "odd_even": true,
    "stubble_ban": false,
    "smog_cannons": true,
    "factory_scaling": 75.0,
    "construction_ban": true
  }
}
```

GET /api/enforcement: Returns prioritized enforcement task tickets.

6. Mathematical Dispersion Models

The simulation core runs on NumPy array vector models, simulating three main environmental principles:

Boundary Layer Trapping: Cool night temperatures trap particulates close to the ground, increasing concentrations in inverse proportion to the mixing height (MH).

Gaussian Wind Dispersion: Biomass stubble fires (F) are carried from the North-West (310°). The plume's impact decreases as the wind direction (WD) deviates from 310°: $ ext{Biomass} = ext{Base} imes e^{- rac{(WD - 310)^2}{2 \cdot 25^2}} imes \left(rac{F}{500} ight) imes \left(rac{15}{WS + 2} ight)$

Diurnal Traffic Curves: Traffic exhaust fluctuates throughout the 24-hour cycle, peaking at rush hour ($t=9, 18$): $ ext{Traffic} = ext{Base} imes \left[1.0 + 0.6 \cdot \sin^2\left(rac{(t - 9) \cdot \pi}{6} ight) ight]$

7. Scalability & Future Roadmap

To scale the platform city-wide:
1. **Live CAAQMS Integration:** Replace mock generation with webhook integrations from the Central Pollution Control Board (CPCB) portals.
2. **Meteorological Feeds:** Bind the dispersion models to real planetary boundary data from weather models (WRF-Chem).
3. **Spatial Indexing:** Deploy timeseries databases (TimescaleDB) and GIS geospatial extensions (PostGIS) to index millions of sensor coordinates across Tier-1 and Tier-2 Indian cities.